AI savviness challenge · Option 1

Design Companion

An AI assistant for the three chores that slow product designers down: turning raw user feedback into prioritised themes, drafting UI copy in the right voice, and sketching genuinely different layout directions.

Rijad Haliti

Senior Product Designer application · Careem

100-word summary

Design Companion is a working prototype that removes three chores from a product designer's week. Paste raw user reviews and it clusters them into severity-rated themes, each with a likely UX cause and one shippable fix. Describe a screen and it writes two copy alternatives, complete with error states and an Arabic-market variant tuned for string expansion and RTL. Describe a feature and it returns three different layout concepts, each rendered as a CSS wireframe. Every prompt forces strict JSON, so the app renders cards rather than chat text. Built in one file with Claude; demo mode needs no key.

Exactly 100 words

01

Feedback Synthesizer

Clusters raw reviews into themes, rates severity, names the likely UX cause, proposes one fix.

In: 12 review lines → Out: 5 rated themes

02

UI Copy Generator

Two copy versions per screen across three tones and two language flavors, error states included.

In: a screen + voice → Out: 2 × 5 strings

03

Layout Brainstormer

Three distinct layout directions, each with hierarchy, components, a trade-off and a wireframe.

In: a feature line → Out: 3 concepts

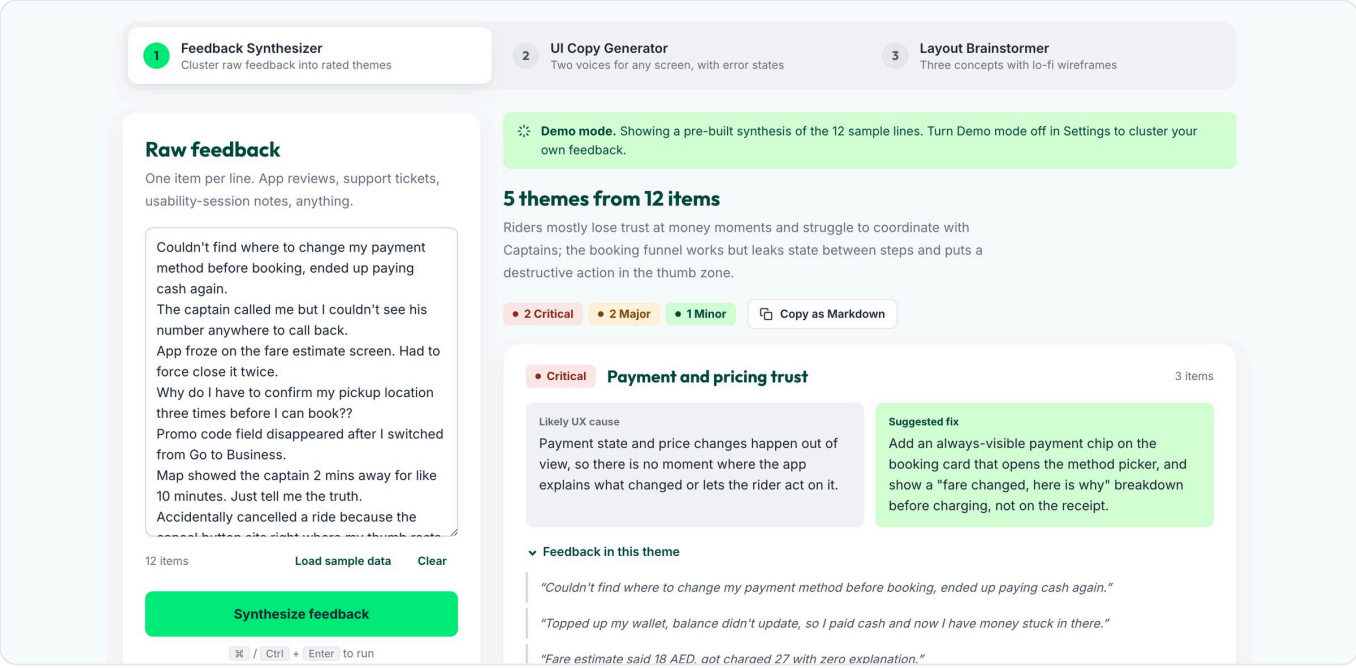
Where to find it

Live prototype	`paste your Netlify or GitHub Pages URL here`
Try it instantly	Demo mode is on by default — no API key, no sign-up. Click through all three tabs and results appear in about a second.
Dataset	Self-created dummy data: 12 fictional ride-hailing review lines, written for this exercise and bundled in the app. No public or confidential data used.
Built with	Claude (Anthropic Messages API) · one self-contained `index.html` · HTML, CSS, vanilla JS · no build step, no dependencies

01 · Feedback Synthesizer

Summarises usability feedback

Paste anything users said — store reviews, support tickets, usability-session notes. The model clusters the lines into themes, rates each one, diagnoses the likely design cause and proposes a single shippable fix. Every source line stays visible under its theme, so the clustering can be audited rather than trusted blindly.



Twelve fictional review lines about a ride-hailing app become five themes, ordered most to least severe. Severity chips are colour-coded; the mint panel is always the fix.

The prompt

```
// system
You are a senior UX researcher embedded in the
design team of Careem… You never pad, never
moralise, and you always respond with a single
valid JSON object and nothing else.

// user
Here are 12 raw feedback items… numbered:
1. …

Cluster these into 3–6 themes. Rules:
- Every item belongs to exactly one theme;
  reference items by number in "feedback_ids".
- "severity": "Critical" blocks a core task,
  loses money or trust, or is a crash; "Major"
  is significant friction on a frequent task;
  "Minor" is cosmetic, rare, or has a workaround.
- "ux_cause": the most likely underlying design
  cause, one specific sentence.
- "fix": one concrete, shippable design change,
  written as an instruction a designer could act
  on tomorrow.
- Order themes from most to least severe.
```

Why it is built this way

Severity is defined, not assumed. Without operational definitions the model drifts between runs and everything becomes "Major". Three sentences of definition make the ratings reproducible.

Items are referenced by number, never re-quoted. The output stays small, the model cannot quietly paraphrase a user, and the UI can show the real lines under each theme.

Cause and fix are separate fields. A designer needs to agree with the diagnosis before accepting the prescription. Merging them hides the reasoning.

What comes back

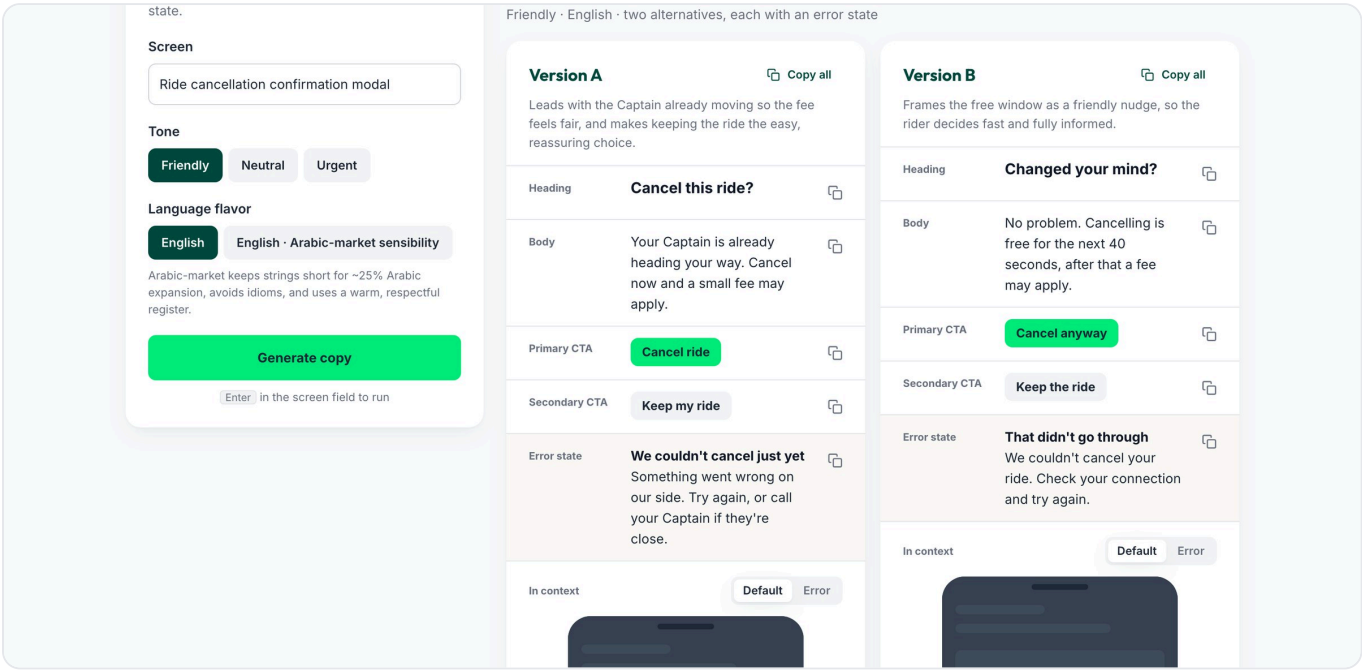
Critical Major Minor

A summary line, then per theme: title, severity, the source line numbers, the UX cause and the fix. The app sorts by severity itself rather than trusting the order, and clamps any out-of-range line number.

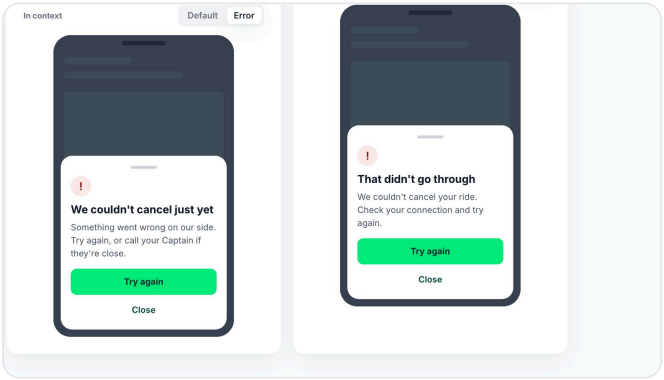
02 · UI Copy Generator

Generates UI copy

Describe a screen, pick a tone and a language flavor. Two alternatives come back — heading, body, primary CTA, secondary CTA and a matching error state — each rendered inside a phone mock so the copy is judged in context rather than as a list of strings. Every field has its own copy button.



Two versions take different angles rather than swapping synonyms: A leads with the Captain already moving, B leads with the free-cancellation window.



The error state is part of every version, previewed in place. The failure path is where copy quality is usually neglected.

The Careem-specific bit

The **Arabic-market** flavor does not translate. It writes English that will survive translation: strings roughly 25% shorter to absorb Arabic expansion, no Western idioms or puns, a warm and respectful register, awareness of prayer times and Ramadan, and drivers always called Captains.

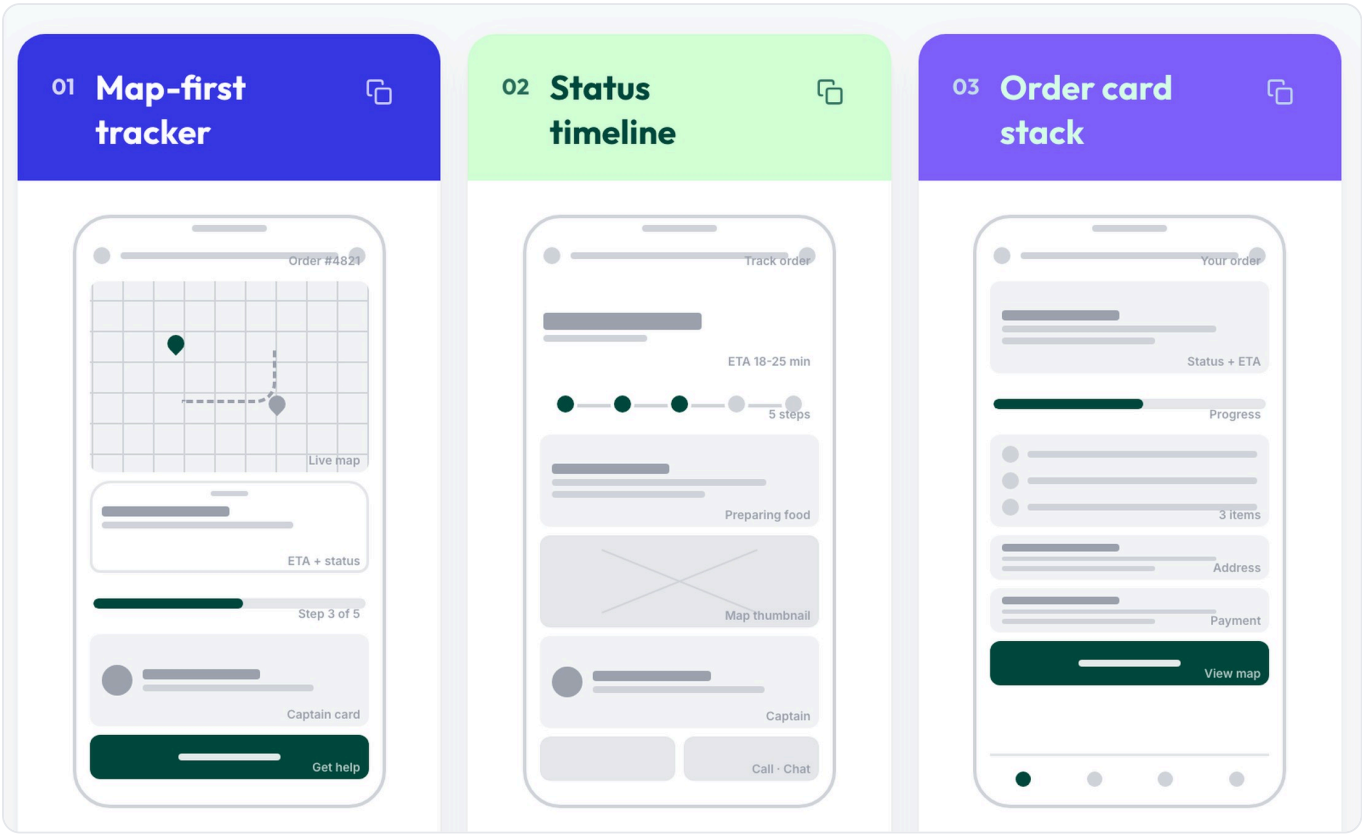
The prompt

```
// system
You are a senior UX writer at Careem. Voice:
clear, warm, human, never robotic or salesy.
Sentence case everywhere. CTAs are short verbs
that describe the outcome ("Cancel ride",
"Keep ride"), never "OK", "Yes" or "No".
Drivers are Captains. Body copy states the
consequence before the choice.

// user
Screen: "Ride cancellation confirmation modal"
Tone: Friendly — warm and conversational…
Language flavor: Arabic-market — …keep every
string roughly 25% shorter so it survives
Arabic expansion and RTL layouts…

Write two distinct alternative versions. A and B
must take different angles (A reassures, B is
direct), not just word swaps.

- heading: max 6 words
- body: max 25 words, states what happens
- primary_cta / secondary_cta: max 3 words
- error_state: what happened + what to do next
- rationale: one sentence, for the design team
```

03 · Layout Brainstormer

Brainstorms layouts

Describe a feature in a line. Three deliberately different directions come back, each with an above-the-fold hierarchy, a component list, one honest trade-off — and a low-fidelity wireframe drawn live in the browser.



Three directions for a food-delivery tracking screen — map-first, timeline-first, card-stack. Each wireframe is CSS boxes built from the block list the model returned, not an image.

The idea worth stealing

The model does not draw — it specifies

Asking an LLM for an image or SVG of a wireframe gives you something unreliable and unreadable. Instead the model picks from a **fixed vocabulary of 18 block types** — map, hero, stepper, chips, sheet, cta, nav and so on — each with a label and a size.

The app renders those blocks as CSS boxes. Output stays deterministic and cheap, fidelity stays honest, and the same JSON could later drive real Figma frames instead of CSS.

header · map · image · hero · text · card · list · chips · stepper · progress · input · cta · buttons · avatar · divider · sheet · tabs · nav

The prompt

```
// user (abridged)
Feature: "Food delivery order tracking screen"
Platform: mobile, portrait, 390 x 844 pt.

Propose exactly 3 genuinely different layout
concepts. They must differ in what dominates
above the fold and in how the hierarchy is
organised, not three variations of one idea.

- name: max 4 words
- hierarchy: 2-3 sentences, top to bottom,
  and why that order
- components: 5-8 named UI components
- tradeoff: one sentence on what this gives up
- wireframe: 5-9 blocks, each
  {"type": header|map|hero|card|stepper|cta|…,
   "label": max 3 words,
   "size": "sm"|"md"|"lg"}
  Use "nav" only last, at most one "map".
```

Forcing a trade-off

Without the `tradeoff` field the model proposes three safe variations of the same layout. Making it name a cost per concept is what pushes the three apart.

How it works

Approach & architecture

One file, no build step, no dependencies. Open it locally or drag it onto a static host and it runs. The interesting decisions are all about making a language model behave like a component rather than a chat box.

Four decisions

1. **JSON, never prose.** Every prompt ends with an explicit schema and "reply with ONLY this JSON". The app parses it and renders styled cards. A chat transcript would not be a design tool.
2. **Demo mode by default.** Hand-written sample outputs, shaped exactly like the real schema, feed the same renderers. A reviewer sees all three tools in thirty seconds with no key, and the demo path proves the UI is schema-driven rather than hardcoded.
3. **Defensive parsing.** Code fences are stripped, severity spellings normalised, out-of-range references dropped, unknown wireframe blocks fall back to a card. Models occasionally return something slightly off; the UI should bend, not break.
4. **Escape everything.** Model output is HTML-escaped before it reaches the DOM. A prototype that renders raw model text into innerHTML is an injection bug waiting to happen.

Under the hood

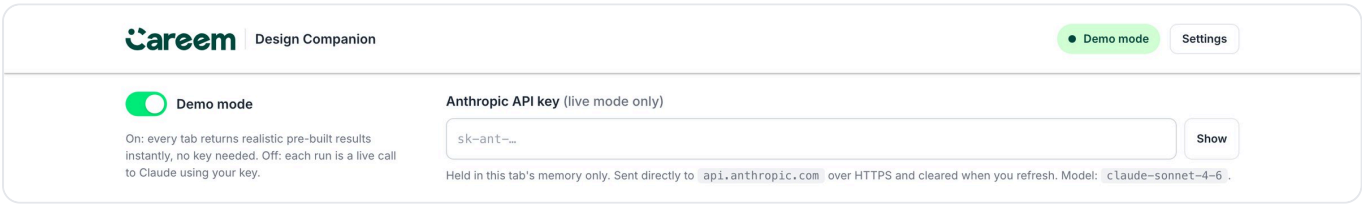
PIECE	CHOICE
Model	Claude, via the Anthropic Messages API, called with `fetch` from the browser
Request	One call per run: a system prompt for role and voice, a user prompt for task, rules and schema
Stack	HTML, CSS and vanilla JS in a single file. No framework, no bundler, no server
Wireframes	Pure CSS boxes. No images, no canvas, no external assets
State	Nothing is stored. The API key lives in a JS variable and dies on refresh

Being straight about the limits

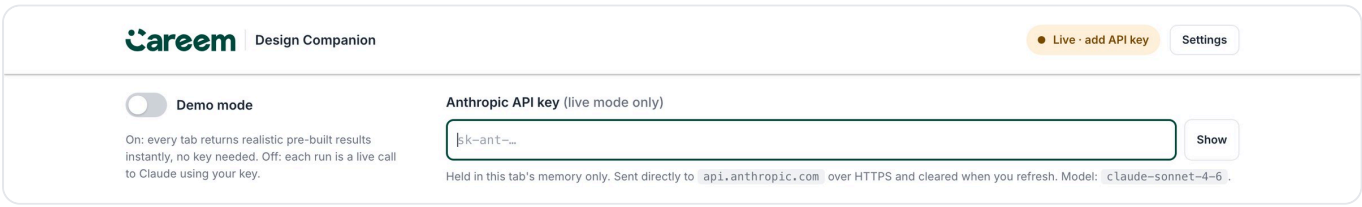
The key is in the browser. Fine for a personal prototype, wrong for production — a real version proxies the call server-side so the key never reaches the client. The prototype says so in its own settings panel.

Demo mode is hardcoded. That is the point: it makes the prototype reviewable without a key. Switch it off, paste a key, and every run is a live model call.

The two modes



Demo mode on — the status pill reads Demo, results are instant and no key is needed. This is what a reviewer sees on first load.



Switched off — the pill turns amber and reads "Live · add API key", and the app refuses to run until a key is pasted. Every run is then a real model call.

Craft notes

Design system & next steps

The prototype is built in Careem's visual language, from a token file rather than by eye. Colour, type, radii and spacing all read from CSS custom properties; there is no hardcoded hex anywhere in the components.

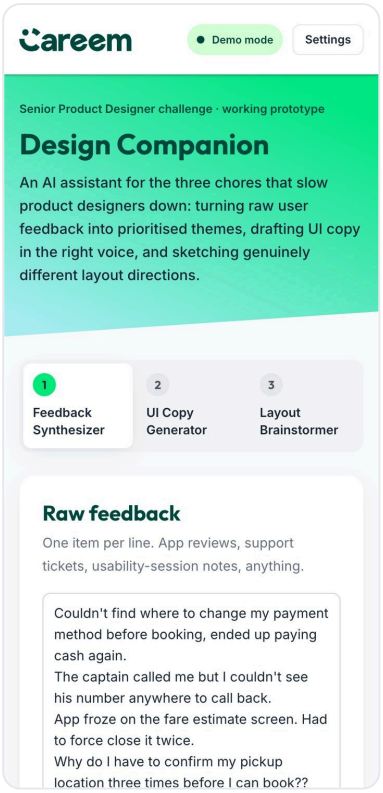
Design system fidelity

- **Two greens, separate jobs.** Neon `#00EB79` for actions, forest `#00493E` for headings — never mixed.
- **Dark text on green.** White on neon green is 1.6:1 and fails. Gray-800 on it is 9.2:1 and passes.
- **Radii by role.** 8px on controls, 16px on cards, never mixed inside one component.
- **Cards stay flat.** Colour and radius carry the hierarchy; shadows are reserved for fixed chrome.
- **The four service colours** — blue, mint, purple, navy — used as a set to separate the three layout concepts.
- **Semantic states stay rare.** Severity reds and ambers are desaturated so they never compete with the brand palette.

Also handled

- **Keyboard.** Arrow keys move between tabs, ⌘/Ctrl+Enter runs the active tool.
- **Screen readers.** Real tab semantics, live regions on results, labelled controls.
- **Loading and failure.** A spinner with skeleton lines, and inline errors for bad key, rate limit, timeout or malformed JSON.
- **Reduced motion** is respected.

Responsive



Result grids use container queries, so cards reflow to the space they actually have rather than to the viewport.

If this became a real internal tool

01 · Move the key server-side

A small proxy function so the key never reaches the browser, plus per-designer rate limits.

02 · Wireframes into Figma

The wireframe JSON already describes blocks and sizes. A plugin could build real frames from the same payload.

03 · Close the loop

Feed real review streams in on a schedule, and let the tool track whether shipped fixes actually moved the theme.

Why I built this one

The brief offered three options and I took the first, but the honest reason is that all three of these chores are ones I actually do. Synthesising feedback is slow and easy to bias. Copy gets written last and reviewed least. Layout exploration collapses to the first idea that looks fine. A tool that produces structured, comparable, auditable output for each of them is more useful to a design team than a chat window — and building it in Careem's own design language felt like the right way to answer a Careem brief.

The brief suggested 30 to 60 minutes. I spent longer, because the question I actually wanted to answer was whether a constrained JSON schema could make a language model produce a wireframe a designer would use. It can, but only if you define the vocabulary for it — that finding is the reason page 4 exists.